

- in general (no python, no c++ only, but pseudocode)
- Moodle quiz after each lecture (optional) → structures, no classes.

ABSTRACT DATA TYPES:

- flexibility (don't change the whole thing)
ex: a struct with 3 fields → I want 1 field only.
- DATA TYPE = ^{DOMAIN =} set of values together with a set of operations.
= INTERFACE
 - ↳ Built-in data types
 - ↳ User-defined data types
ex: struct for representing a fraction.
- + ABSTRACT = no testing, no implementations.
↳ focus on WHAT, not HOW.

Ex. of user-defined ADT:

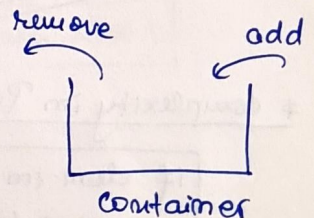
Date ADT: → year
 → month
 → day } = DOMAIN (set of values)

difference (d₁, d₂) = INTERFACE (set of operations)

C++ CONTAINER⁴:

container = collection of data, ≠ types.

- ↳ you can add, remove
- ↳ access elements using ITERATOR.



Python, ... "COLLECTION⁴":

Ex. of containers:

CONTAINERS

LIST

- not unique
- elements have positions
- any element is accessible
- simple elements

DICTIONARY

- keys are unique
- no positions (key ≠ position)
- any element is accessible (by key)
- (key-val) elements

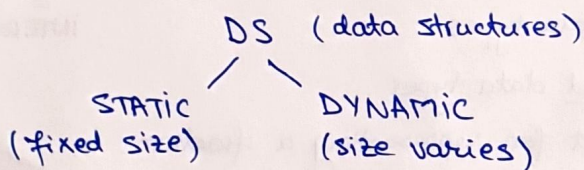
ADT → I don't know anything about implementation, but I know enough info about domain to implement it.
other things

Roman Numerals

→ Key-value pairs → dictionary

→ but I need $\begin{cases} \text{Key} \rightarrow \text{value} \\ \text{value} \rightarrow \text{Key} \end{cases}$ → still a dictionary solves the pb.

Implementation = how to STORE data into computer.



DS vs. DT vs. ADT

- array, stack, tree
- how data is stored
- implementation

- int, char, float
- basic kind of data

- stack (push, pop) ^{impl.} → array, list.
- logical model
- defines what operations, not how implement
- behaviour (abstract)

≠ complexity in Python:

```
if elem in container:
    print("Found")
```

→ list
→ dict

@ → non-standard instructions (#TODO) "↑" → pointer.

"//" → one-line comments

"read" → reading data

"print" → printing data

"←" → assignment

"==" → testing the equality of two variables.

!Indexing from 1 in pseudocode!

CONDITIONAL INSTRUCTIONS:

```
if condition then
  @instr
else
  @instr
end-if
```

FOR LOOPS:

```
for i ← init, final, step execute
  @instr
end-for
```

WHILE LOOPS:

```
while condition execute
  @instr
end-while
```

SUBALGORITHMS: → subprograms that don't return a value.

```
subalgorithm name (formal param list) is:
  @instr
end-subalgorithm
```

? ↗ formal
? ↘ actual

calling a subalgorithm: `name (actual param list)`

FUNCTIONS: → subprograms that return a value

```
function name (formal param list) is:
  @instr
  name ← r // syntax used to return the value r.
end-function
```

calling a function: `result ← name (actual param list)`

Define a variable of type X

ex: $i : \text{Integer} \rightarrow$ variable i of type Integer.

$a : T[] \rightarrow$ array a of type T . (don't know the size)

$a : T[Nr] \rightarrow 1 \dots Nr$ (I know the size)

↳ indexing from $1, Nr \rightsquigarrow [1, Nr]$

STRUCT:

```
Array:
  m: Integer
  elems: T[]
```

} two fields → m of type Integer
 → array elems of type T .

Through `var: Array` we can access the fields

- `var.m`
- `var.elems`
- `var.elems[i]`

pointers = variables whose value is an address.

ex: $p: \uparrow \text{Integer} \rightarrow p$ is a variable whose value is the address of a mem. loc. where an Integer value is stored.

$\text{allocate}(p) \rightarrow$ allocation operation

$\text{free}(p) \rightarrow$ de-allocation

$\text{NIL} \rightarrow$ invalid address.

Name of a parameter in the specification = its value.

$i \in T \rightarrow$ the value of variable i belongs to the domain of type T .

ex: pre: $x \in \text{Integer}$

post: $x' = x - 1$ (Not $x = x - 1$).

$\text{TElem} \rightarrow$ the elements of a container ADT are of generic type.

$\hookrightarrow$ its interface:

- assignment ($e_1 \leftarrow e_2$)

pre: $e_1, e_2 \in \text{TElem}$

post: $e_1' = e_2$

- equality test ($e_1 = e_2$)

pre: $e_1, e_2 \in \text{TElem}$

post:

$\text{equal} \leftarrow \begin{cases} \text{True, if } e_1 \text{ equals } e_2 \\ \text{False, otherwise} \end{cases}$
(result)

$\text{TComp} \rightarrow$ compared/ordered

- compare (e_1, e_2)

pre: $e_1, e_2 \in \text{TComp}$

post:

$\text{compare} \leftarrow \begin{cases} \text{True, if } e_1 < e_2 \\ \text{False, if } e_1 > e_2 \end{cases}$
(result)

$e_1 \leq e_2$
 $e_1 = e_2$
 $e_1 > e_2$

} instead of calling the "compare" function.

AK model: simplified model of how computers work.

Nr. of steps = a function that depends on the SIZE of input data.

$$T(m) = 1 + 2 * m * (1 + m + 1) + 1 = 2(m^2) + 4m + 2 = \text{nr. of steps.}$$

↳ not the complexity! (intermediate step).

↳ grows as fast as m^2 does (still not the complexity)

↓
"leading term"

Complexity = HOW DO THE NR. OF STEPS CHANGE if the value of m increases
= order of growth.

O-notation UPPER BOUND

For a given function $g(m)$ we denote by $O(g(m))$ the set of functions:

$$O(g(m)) = \{f(m) : \exists c > 0 \text{ and } m_0 \text{ s.t. } 0 \leq f(m) \leq c \cdot g(m), (\forall) m \geq m_0\}$$

$$f(m) \in O(g(m)) \text{ if } \lim_m \frac{f(m)}{g(m)} \text{ is ct. (not } \infty)$$

ex: $T(m) = m^2 + 2m + 2$

$$T(m) = O(m^2) \text{ because } T(m) \leq c \cdot m^2 \text{ for } c=2 \text{ and } m \geq 3.$$

? $T(m) = O(m^3)$ because $\lim_m \frac{T(m)}{m^3} = 0 = \text{ct.}$

Ω -notation LOWER BOUND

$$\Omega(g(m)) = \{f(m) : \exists c > 0 \text{ and } m_0 \text{ s.t. } 0 \leq c \cdot g(m) \leq f(m), (\forall) m \geq m_0\}$$

$$f(m) \in \Omega(g(m)) \text{ if } \lim_m \frac{f(m)}{g(m)} \text{ is } \infty \text{ or a } \neq 0 \text{ ct.}$$

Θ -notation ACTUAL VALUE.

$$\Theta(g(m)) = \{f(m) : \exists c_1, c_2 > 0 \text{ and } m_0 \text{ s.t. } 0 \leq c_1 \cdot g(m) \leq f(m) \leq c_2 \cdot g(m), (\forall) m \geq m_0\}$$

$$f(m) \in \Theta(g(m)) \text{ if } \lim_m \frac{f(m)}{g(m)}$$

Sum of all even nr. in array

length of array matters!

first occurrence of a nr. in array

length does not matter!

↓
position of elem?

↓
AC, BC, WC

↓
 Θ

BC: nr. of steps is min.

WC: nr. of steps is max.

AC: average of all possible cases.

↳ formula: $\sum_{I \in \text{IED}} P(I) \cdot E(I)$

one
input data

set of all
inputs

$P(I)$: probability of having I as input.

$E(I)$: nr. of operations performed by the alg. for I .
(ex: how many times we perform the loop?).

I = all the arrays where the first nr = the searched one.

I = " — " — the second nr = " — "

⋮

I = " — " — the first $(m-1)$ ^{el.} are not what we searched, but the last one is.

I = " — " — don't contain the el. we were looking for.

$P(I)$ is the same for every I .

↳ here, $\frac{1}{m+1}$.

$$2) T(m) = \frac{1}{m+1} \cdot \underbrace{\sum_{i=1}^m i}_{\frac{m(m+1)}{2}} + \frac{m}{m+1} = \frac{m(m+1)}{2(m+1)} + \frac{m}{m+1} \in \Theta(m).$$

Recursive Binary Search.

$$\bullet T(m) = \begin{cases} 1, & \text{if } m \leq 1 \\ T(\frac{m}{2}) + 1, & \text{otherwise.} \end{cases} \quad \leftarrow \text{recursive formula of the solution}$$

$\downarrow$
good to always have sth. along $T(\dots)$...

- Suppose $m = 2^k$, rewrite second branch.

$$T(2^k) = T\left(\frac{2^k}{2}\right) + 1 = T(2^{k-1}) + 1$$

$$T(2^{k-1}) = T(2^{k-2}) + 1$$

$\vdots$

$$T(2^1) = T(2^0) + 1$$

(+)

$$T(2^k) = \underbrace{T(2^0)}_{\substack{\parallel \\ 1.}} + \underbrace{1 + \dots + 1}_{k \text{ 1's}} = 1 + k.$$

! For non-recursive functions, $(+1)$ does not influence.
recursive functions, $(+1)$ is important to have.

$$m = 2^k \Rightarrow k = \log_2 m.$$

$$T(2^k) = 1 + k$$

$$2) T(m) = 1 + \log_2 m \in \Theta(\log_2 m)$$

$\nwarrow$
choose the bigger?